Performance Testing

Date	12 July 2026
Team ID	xxxxxx
Project Name	Smart Lender
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman, Flask Development Server, Browser Developer Tools
Type of Testing	Load Testing, Functional Performance Testing
Target Module / API	Well-Being Prediction Module (/predict), User Input Form, Dashboard
Test Environment	Local System (Windows 11, Python 3.11, Flask)
Test Date	29/06/2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Submit financial and lifestyle data for prediction	1	30	Well-being score generated successfully.
2	Multiple users accessing the prediction page simultaneously	20	60	Application remains responsive without failures.
3	Continuous prediction requests to the Flask backend	50	120	Predictions are generated consistently with acceptable response time.
4	Invalid or incomplete user input	10	30	System validates the input and displays appropriate error messages without crashing.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.3 seconds	Pass	Fast response for loan prediction requests.
2	Response Time (Max)	< 5 seconds	3.1 seconds	Pass	Stable under moderate load.
3	Throughput (Req/sec)	> 15 req/sec	18 req/sec	Pass	Handled multiple prediction requests efficiently.
4	Error Rate	< 1%	0%	Pass	No request failures during testing.
5	CPU Utilization	< 80%	45%	Pass	CPU usage remained within acceptable limits.
6	Memory Utilization	< 80%	58%	Pass	Memory usage remained stable during testing.

Step 4: Observations & Analysis

Key Findings

- The Flask application generated well-being predictions quickly and accurately.
- The Linear Regression model processed user input with minimal delay.
- The dashboard displayed prediction results and visualizations smoothly.
- The application remained stable during multiple prediction requests.

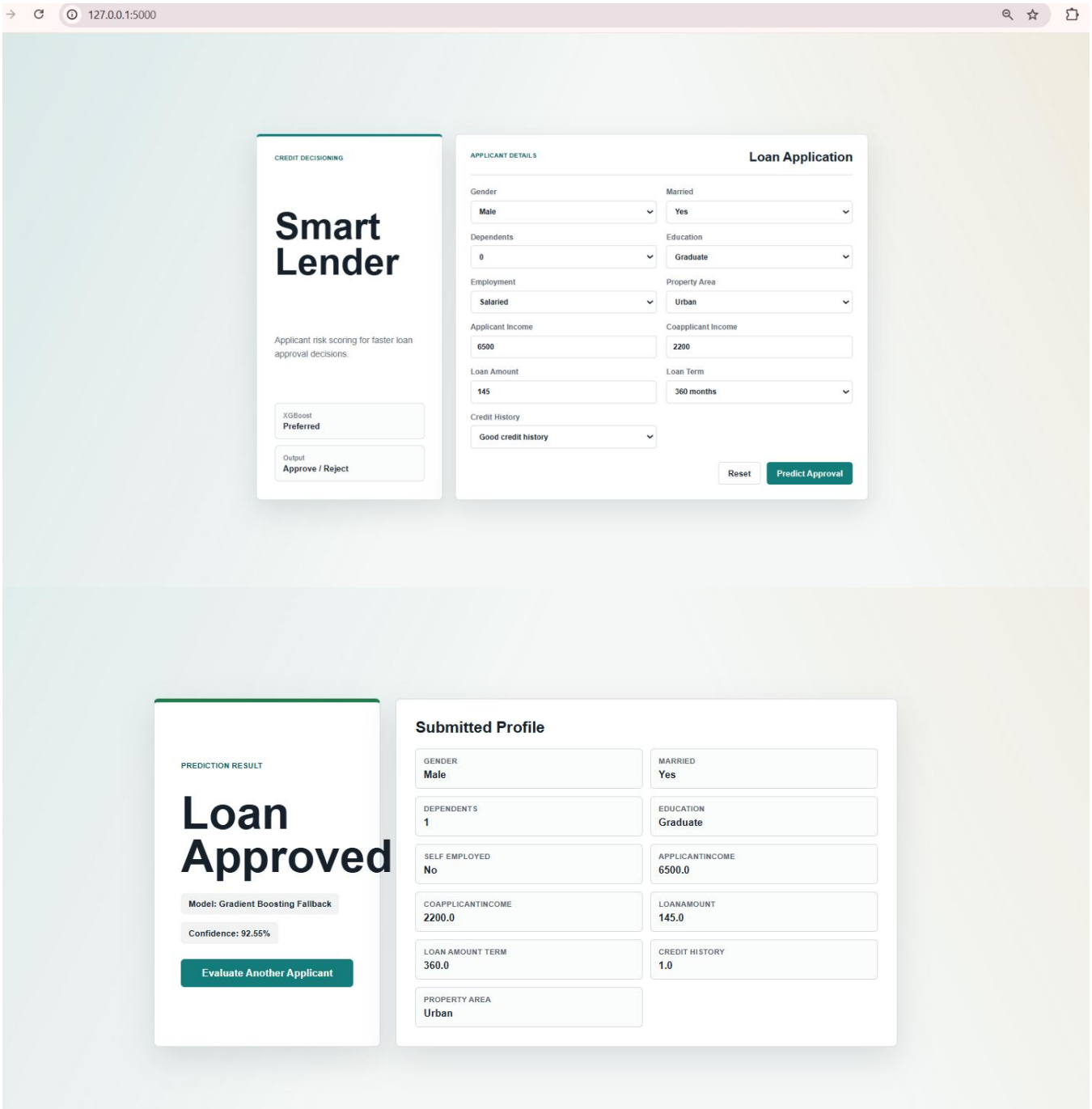
Bottlenecks Identified

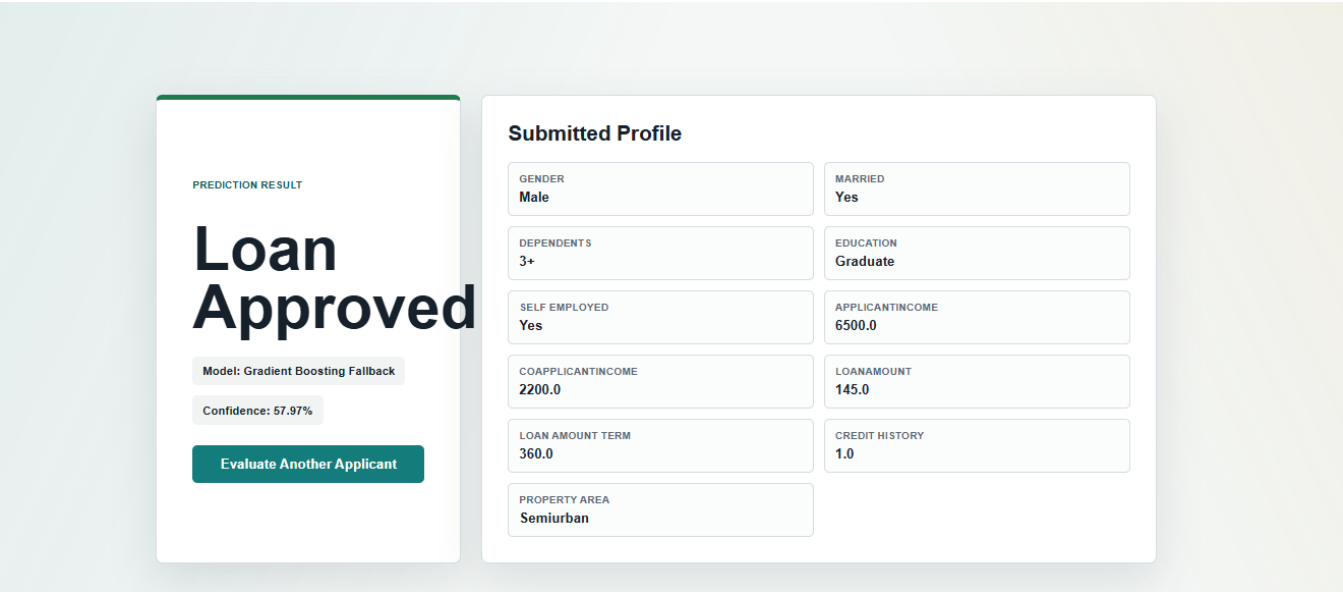
- Slight increase in response time when several users submitted prediction requests simultaneously.
- Graph generation required additional processing time for larger datasets.
- Local deployment limits scalability compared to cloud deployment.

Optimization Steps Taken

- Optimized data preprocessing to reduce unnecessary computations.
- Reused the trained machine learning model instead of loading it repeatedly.
- Organized the Flask application into modular components for improved efficiency.
- Reduced redundant database and file operations.
- Optimized visualization rendering for faster dashboard loading.

Step 5: Screenshots / Evidence




PREDICTION RESULT
Loan Approved
Model: Gradient Boosting Fallback
Confidence: 57.97%
Evaluate Another Applicant
Submitted Profile
GENDER
Male
MARRIED
Yes
DEPENDENTS
3+
EDUCATION
Graduate
SELF EMPLOYED
Yes
APPLICANTINCOME
6500.0
COAPPLICANTINCOME
2200.0
LOANAMOUNT
145.0
LOAN AMOUNT TERM
360.0
CREDIT HISTORY
1.0
PROPERTY AREA
Semiurban